

Universidad Autónoma de Sinaloa

Facultad de ingeniería Mochis



Licenciatura en Ingeniería de Software

Materia:

Administración de Sistemas

Tarea:

Tarea 4: SSH

Maestro:

Herman Geovany Ayala Zuñiga

Alumno:

Edrik Eduardo Camacho Rojo

1. Objetivo

Implementar y asegurar el servicio de acceso remoto SSH en los servidores Linux (OpenSSH-Server) y Windows (característica OpenSSH Server), habilitando su inicio automático y la apertura de puertos en los firewalls correspondientes. A partir de este punto, toda la administración de los servidores —incluida la de las prácticas anteriores de DHCP y DNS— se realiza exclusivamente vía sesión SSH remota, sin volver a tocar la consola física o virtual de las máquinas.

2. Entorno Técnico

Nodo	Rol / Servicio	Dirección IP
Ubuntu Server 24.04 (CLI)	OpenSSH-Server + DHCP + DNS (BIND9)	192.168.100.10
Windows Server 2022	OpenSSH Server + rol DHCP + rol DNS	192.168.100.20
Linux Mint (cliente)	Cliente SSH / receptor DHCP-DNS	192.168.100.30

Cliente de administración: terminal nativa (ssh), con opción de cliente gráfico (MobaXterm / PuTTY) para las evidencias solicitadas.

3. Arquitectura de Software

A partir de esta práctica, la lógica de todos los scripts (Prácticas 1, 2 y 3, más el nuevo módulo de SSH) se separa en bibliotecas de funciones cargadas por un único script principal (menú), en vez de mantener código lineal repetido en cada archivo.

3.1 Mapa de Archivos del Repositorio

```

tarea4-ssh/
├── README.md
├── linux/
│   ├── main.sh                # Punto de entrada (menu) - Linux
│   └── lib/
│       ├── funciones_comunes.sh # verificar_root, instalar_paquete,
│       │                               # validar_ip, habilitar_servicio
│       ├── funciones_ssh.sh      # Instalacion y hardening de SSH
│       ├── funciones_dhcp.sh    # Logica de la Practica 2 (DHCP)
│       └── funciones_dns.sh     # Logica de la Practica 3 (DNS)
├── windows/
│   ├── Main.ps1               # Punto de entrada (menu) - Windows
│   └── lib/
│       ├── FuncionesComunes.ps1
│       ├── FuncionesSSH.ps1
│       ├── FuncionesDHCP.ps1
│       └── FuncionesDNS.ps1
└── docs/
    └── Tarea4_SSH.docx
  
```

3.2 Cuadro Comparativo de Refactorización

Ejemplo tomado de la lógica de instalación de paquetes, usada originalmente en las Prácticas 2 y 3 de forma repetida en cada script.

Antes (lineal, repetido en cada práctica)	Después (modular, con funciones)
<pre># dentro de tarea2_dhcp.sh apt update apt install -y isc-dhcp-server # dentro de tarea3_dns.sh apt update apt install -y bind9</pre>	<pre># funciones_comunes.sh function instalar_paquete() { local paquete="\$1" apt update -qq apt install -y "\$paquete" } # main.sh source ./lib/funciones_comunes.sh instalar_paquete "isc-dhcp-server" instalar_paquete "bind9"</pre>

El mismo patrón se aplicó en PowerShell: la validación de administrador, la instalación de roles (Install-WindowsFeature) y la verificación de IPs, que antes se repetían en cada script (Tarea1.ps1, Tarea2_DHCP.ps1, Tarea3_DNS.ps1), ahora viven una sola vez en FuncionesComunes.ps1 y se importan con el operador de punto (.).

3.3 Guía de Conexión SSH

Pasos para administrar cada servidor de forma remota desde el equipo cliente, una vez completada la instalación local de SSH.

Servidor Linux (Ubuntu Server 24.04)

- Verificar que el servicio esté activo: `sudo systemctl status ssh`
- Desde el cliente (terminal): `ssh usuario@192.168.100.10`
- Aceptar la huella digital (fingerprint) la primera vez que se conecta.
- Introducir la contraseña del usuario para autenticarse.
- Con cliente gráfico (MobaXterm/PuTTY): Host = 192.168.100.10, Puerto = 22, protocolo SSH.

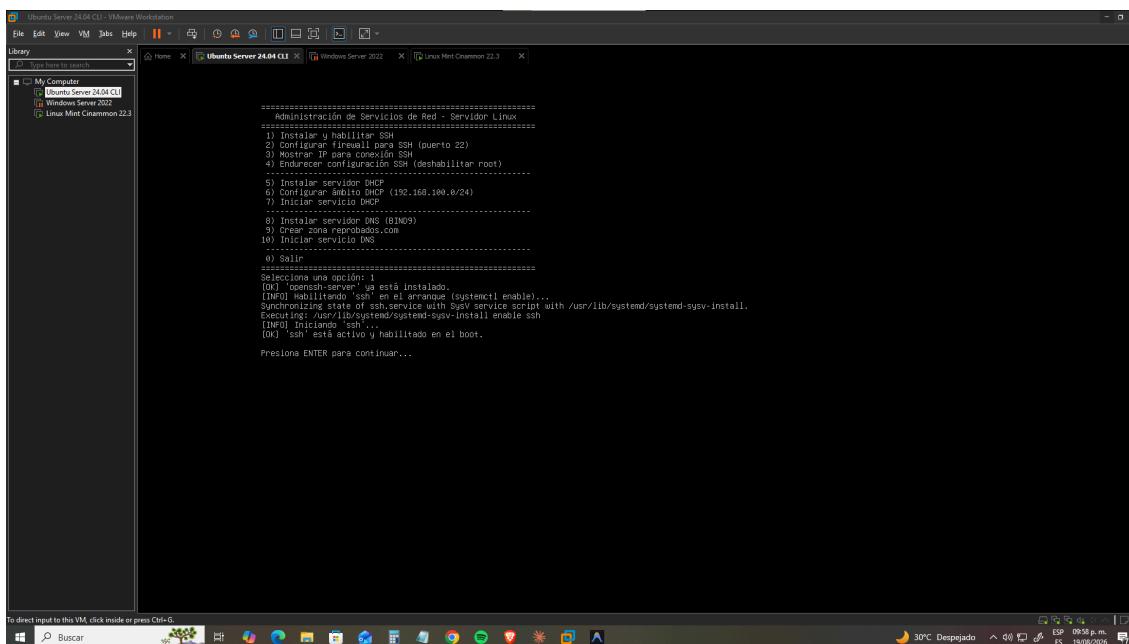
Servidor Windows (Windows Server 2022)

- Confirmar que el servicio 'sshd' esté en ejecución: `Get-Service sshd`
- Confirmar la regla de firewall: `Get-NetFirewallRule -Name OpenSSH-Server-In-TCP`
- Desde el cliente (terminal): `ssh usuario@192.168.100.20`
- Con cliente gráfico (MobaXterm/PuTTY): Host = 192.168.100.20, Puerto = 22, protocolo SSH.
- La sesión abre por defecto PowerShell (configurado en DefaultShell).

A partir de la conexión exitosa a ambos servidores, cualquier tarea de administración adicional (ajustes de DHCP, zonas DNS, revisión de logs) se ejecuta dentro de la sesión SSH, nunca desde la consola de VMware.

4. Implementación de SSH

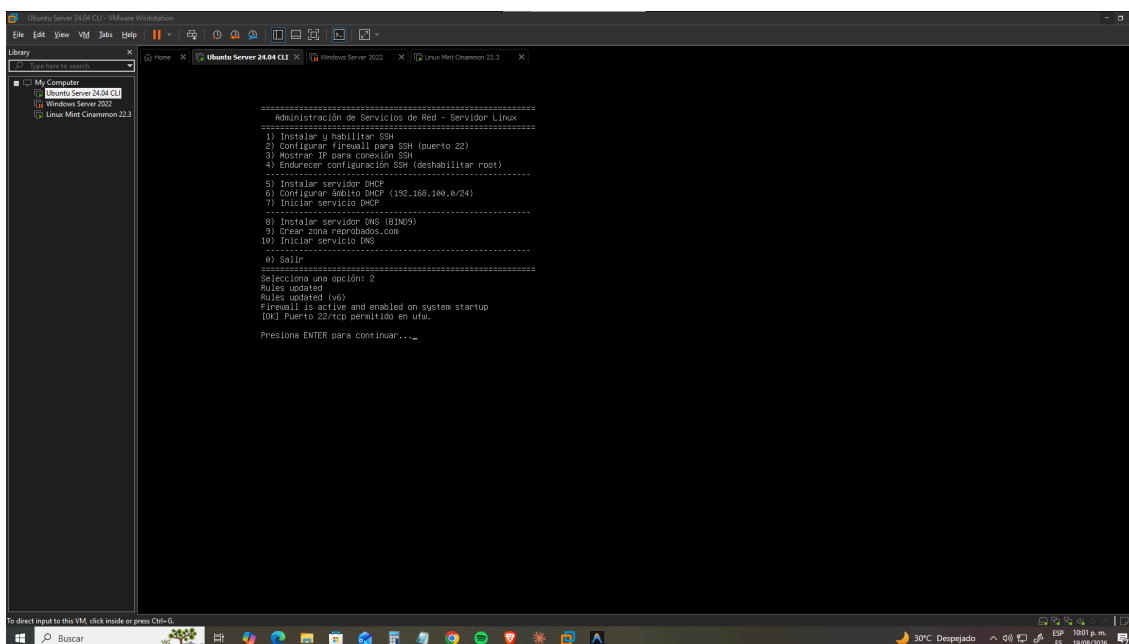
4.1 Linux — instalación y arranque en boot



```

=====
Administración de Servicios de Red - Servidor Linux
=====
1) Instalar y habilitar SSH
2) Configurar firewall para SSH (puerto 22)
3) Mostrar IP para conexión SSH
4) Endurecer configuración SSH (deshabilitar root)
=====
5) Instalar servidor DHCP
6) Configurar ámbito DHCP (192.168.100.0/24)
7) Iniciar servicio DHCP
=====
8) Instalar servidor DNS (BIND9)
9) Crear zona reprobados.com
10) Iniciar servicio DNS
=====
0) Salir
=====
Selección una opción: 1
[OK] 'openssh-server' ya está instalado.
[INFO] Habilitando 'ssh' en el arranque (systemctl enable)...
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
[INFO] Iniciando 'ssh'...
[OK] 'ssh' está activo y habilitado en el boot.
Presiona ENTER para continuar...
  
```

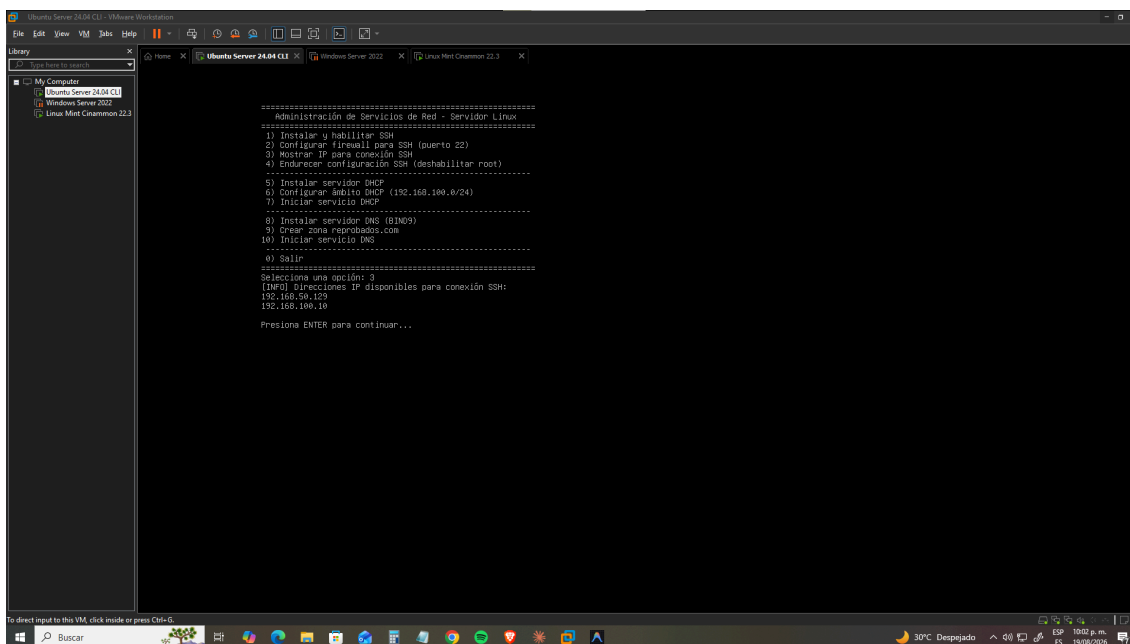
La función `instalar_ssh()` de funciones `ssh.sh` instala `openssh-server` y llama a `habilitar_servicio("ssh")`, que ejecuta `systemctl enable ssh` y `systemctl restart ssh`, verificando con `systemctl is-active` que el servicio quedó activo.



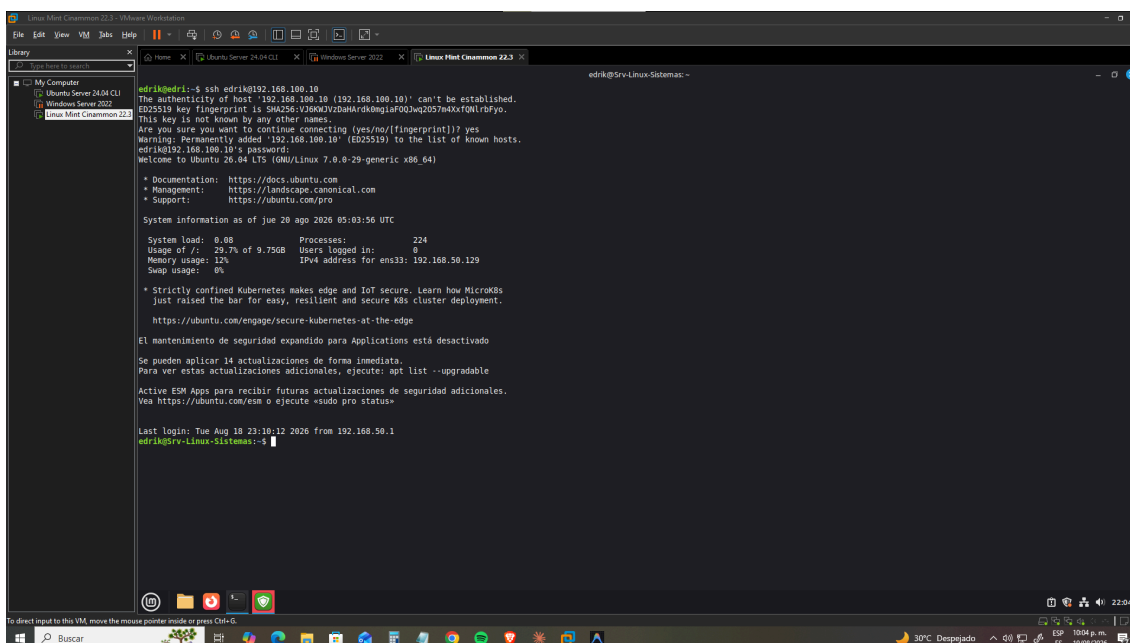
```

=====
Administración de Servicios de Red - Servidor Linux
=====
1) Instalar y habilitar SSH
2) Configurar firewall para SSH (puerto 22)
3) Mostrar IP para conexión SSH
4) Endurecer configuración SSH (deshabilitar root)
=====
5) Instalar servidor DHCP
6) Configurar ámbito DHCP (192.168.100.0/24)
7) Iniciar servicio DHCP
=====
8) Instalar servidor DNS (BIND9)
9) Crear zona reprobados.com
10) Iniciar servicio DNS
=====
0) Salir
=====
Selección una opción: 2
Rules updated
Rules reloaded (v6)
Firewall is active and enabled on system startup
[OK] Puerto 22/tcp permitido en ufw.
Presiona ENTER para continuar...
  
```

Configuración de Firewall

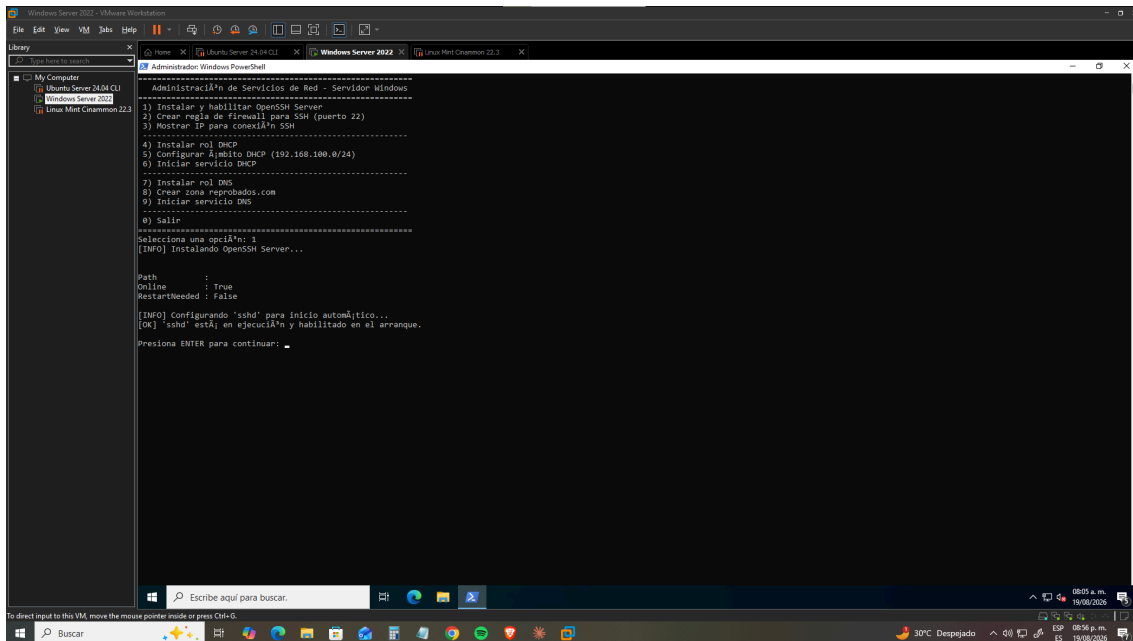


Direcciones IP disponibles



Prueba en cliente Linux Mint haciendo SSH hacia Ubuntu Server

4.2 Windows — habilitación de servicio y firewall



```

Administración de Servicios de Red - Servidor Windows

1) Instalar y habilitar OpenSSH Server
2) Crear regla de firewall para SSH (puerto 22)
3) Mostrar IP para conexión SSH
4) Instalar rol DHCP
5) Configurar servicio DHCP (192.168.100.0/24)
6) Iniciar servicio DHCP
7) Instalar rol DNS
8) Crear zona reprobados.com
9) Iniciar servicio DNS
0) Salir

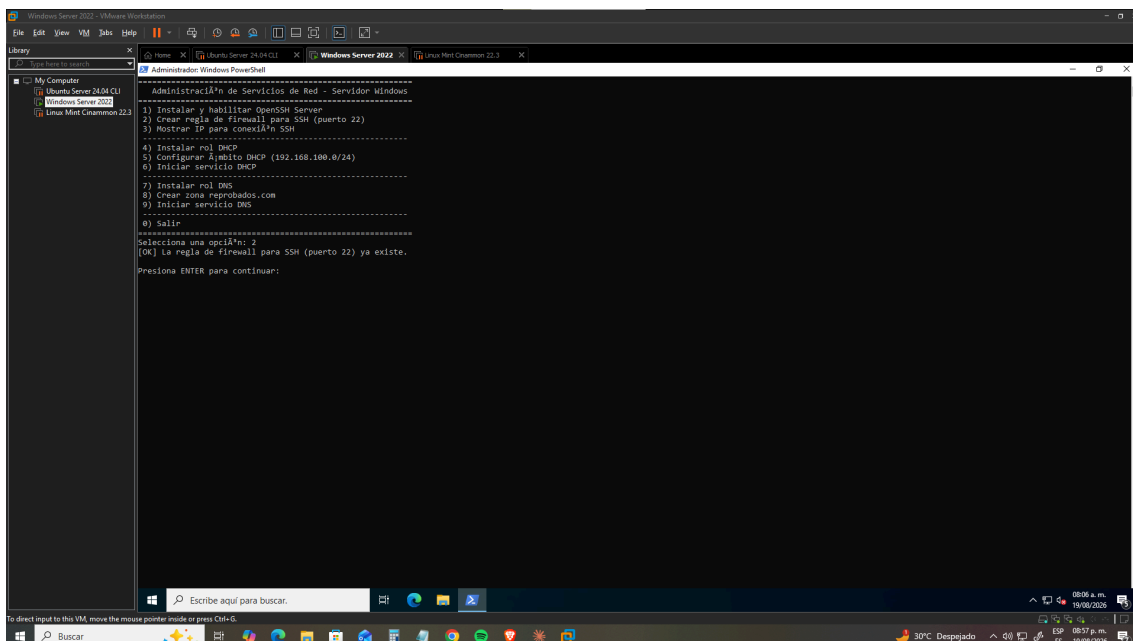
=====
Seleccione una opción: 1
[INFO] Instalando OpenSSH Server...

Path      :
Online    : True
RestartNeeded : False

[INFO] Configurando 'sshd' para inicio automático...
[OK] 'sshd' está en ejecución y habilitado en el arranque.

Presione ENTER para continuar:
  
```

La función `Install-SshServer` agrega la capacidad `OpenSSH.Server` con `Add-WindowsCapability` y configura el servicio `sshd` como automático.



```

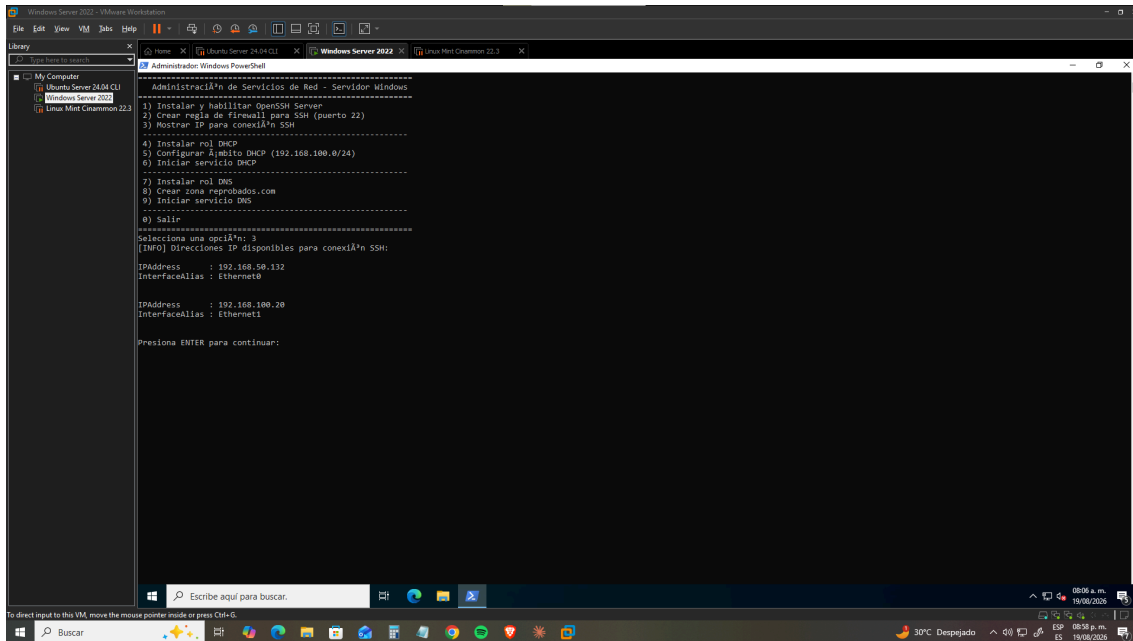
Administración de Servicios de Red - Servidor Windows

1) Instalar y habilitar OpenSSH Server
2) Crear regla de firewall para SSH (puerto 22)
3) Mostrar IP para conexión SSH
4) Instalar rol DHCP
5) Configurar servicio DHCP (192.168.100.0/24)
6) Iniciar servicio DHCP
7) Instalar rol DNS
8) Crear zona reprobados.com
9) Iniciar servicio DNS
0) Salir

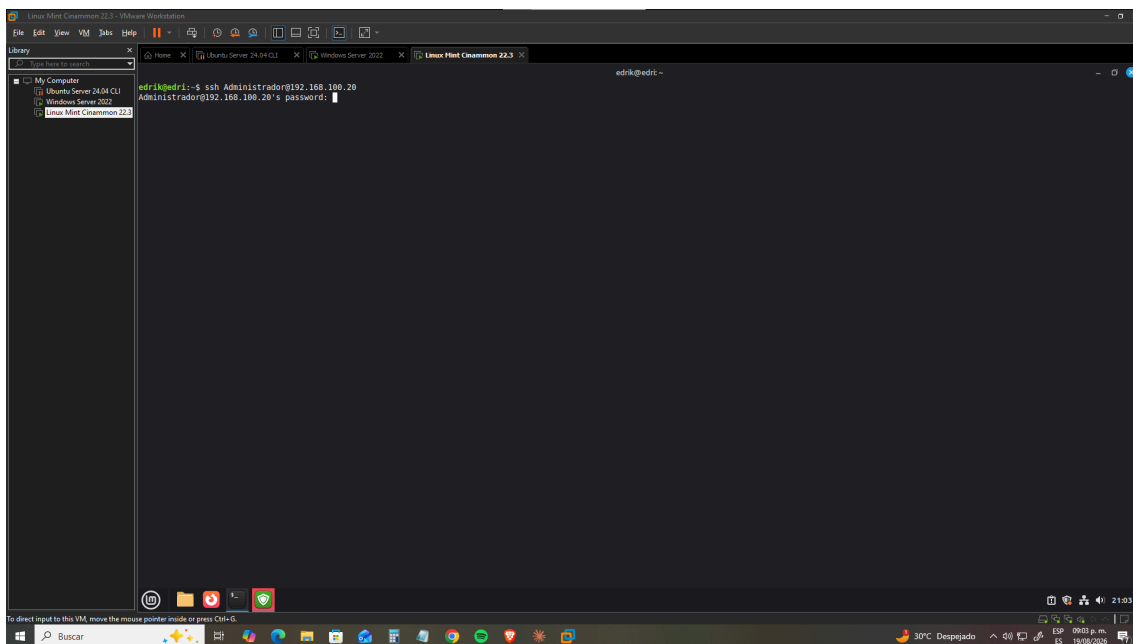
=====
Seleccione una opción: 2
[OK] La regla de firewall para SSH (puerto 22) ya existe.

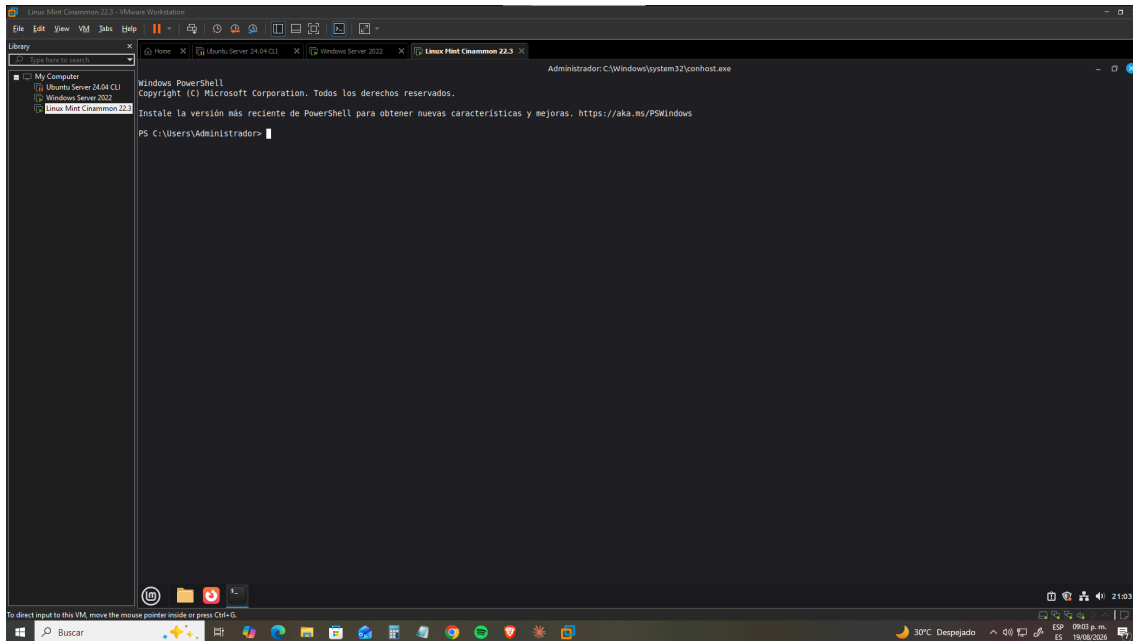
Presione ENTER para continuar:
  
```

`New-ReglaFirewallSSH` crea la regla `OpenSSH-Server-In-TCP` para permitir tráfico entrante TCP en el puerto 22.



Direcciones IP disponibles





Prueba en cliente Linux Mint haciendo ssh dentro de Windows Server

4.3 Hito crítico

Una vez validada la conexión SSH a ambos servidores, no se vuelve a usar la consola física/virtual: toda configuración posterior (incluyendo cambios a DHCP y DNS) se documenta con evidencias tomadas dentro de la sesión SSH.

5. Sincronización con GitHub

Cada etapa de la refactorización se registra en el historial de commits del repositorio, usando mensajes descriptivos que reflejan el proceso de reingeniería:

```
git add linux/lib/funciones_comunes.sh
git commit -m "Refactor: extraer funciones comunes (root, paquetes, IP)"

git add linux/lib/funciones_dhcp.sh linux/lib/funciones_dns.sh
git commit -m "Refactor: migracion de logica DHCP y DNS a funciones modulares"

git add linux/lib/funciones_ssh.sh linux/main.sh
git commit -m "Feature: modulo SSH + menu principal main.sh"

git add windows/
git commit -m "Refactor: modularizacion equivalente en PowerShell (Windows)"
```


6. Autoevaluación contra la Rúbrica

Criterio	Evidencia	Peso
Funcionalidad SSH	Conexión validada desde el cliente a ambos servidores; puertos 22 abiertos en ambos firewalls.	30%
Refactorización	Prácticas 1, 2 y 3 migradas a funciones y archivos modulares (lib/), sin código lineal repetido.	40%
GitHub	Historial de commits descriptivo del proceso de reingeniería (ver sección 5).	15%
Documentación	Mapa del repositorio, cuadro comparativo y evidencias de administración vía SSH.	15%